



UNIVERSIDAD
TECNOLÓGICA
METROPOLITANA
del Estado de Chile

INF8090 - COMP. PARALELA Y DISTRIBUIDA
INGENIERÍA CIVIL EN CIENCIA DE DATOS

PAUTA DE ELABORACIÓN Y PRESENTACIÓN:

*“Proyecto Semestral de Computación Paralela y
Distribuida”*

Dr. Ing. Michael Miranda Sandoval

02 de julio de 2026

PAUTA DE ELABORACIÓN Y PRESENTACIÓN:

Identificación del grupo:

Nombre: _____ Email: _____

Nombre: _____ Email: _____

Nombre: _____ Email: _____

- **Actividad:** Proyecto semestral de la asignatura.
- **Modalidad:** Grupal, con máximo 3 estudiantes, salvo indicación distinta del docente.
- **Contexto del curso:** computación paralela y distribuida aplicada a ciencia de datos.
- **Productos principales:** informe final, código o prototipo reproducible, evidencia experimental y presentación final.
- **Duración de la presentación:** máximo 15 minutos por grupo.
- **Fechas clave de presentación:** plazo máximo jueves 16 de julio de 2026. Se permite **presentación anticipada el jueves 09 de julio de 2026**, previo aviso al profesor vía correo electrónico.
- **Clase de orientación:** el jueves 09 de julio de 2026 habrá clases para orientar a estudiantes que requieran apoyo en la finalización de sus proyectos.
- **Evaluación asociada:** Proyecto Semestral, según calendario y ponderación definidos por el programa de la asignatura.

1. Propósito de la pauta

Esta pauta orienta la elaboración del proyecto semestral de Computación Paralela y Distribuida. Su objetivo es que cada grupo formule, implemente y evalúe una solución computacional aplicada a ciencia de datos donde el uso de paralelismo, concurrencia, distribución o una arquitectura híbrida tenga una justificación técnica real.

El proyecto no debe limitarse a ejecutar una herramienta o a paralelizar código de forma mecánica. Debe demostrar diagnóstico del problema, diseño de una estrategia, comprensión de las restricciones de datos y arquitectura, implementación defendible, medición experimental, interpretación de resultados y discusión honesta de limitaciones.

Relación con la progresión del curso:

- Semanas iniciales: fundamentos, motivación, modelos, taxonomías, speedup, eficiencia y escalabilidad.
- Desarrollo intermedio: concurrencia en Python, procesos, hilos, GIL, diseño de algoritmos paralelos, particionamiento, granularidad y dependencias.
- Desarrollo avanzado: OpenMP, memoria compartida, sincronización, comunicación distribuida, MPI, balanceo de carga, tolerancia a fallas, estructuras de datos paralelas y sistemas híbridos.
- Cierre: aplicaciones en ciencia de datos, profiling, análisis experimental, informe final y presentación técnica.

2. Resultado esperado del proyecto

Al finalizar el proyecto, el grupo debe ser capaz de demostrar que comprende un problema aplicado y que puede defender una decisión de diseño computacional con evidencia. El producto final debe

responder, como mínimo, las siguientes preguntas:

1. **¿Qué problema se intenta resolver?** Debe existir un problema claro, delimitado y pertinente para ciencia de datos.
2. **¿Por qué el problema justifica paralelismo o distribución?** Debe existir una razón concreta: volumen de datos, costo computacional, latencia, throughput, memoria, procesamiento por lotes, simulación, búsqueda, grafos, embeddings, streaming u otra restricción defendible.
3. **¿Qué estrategia se implementó?** Se debe explicar el modelo de paralelismo, el particionamiento, las dependencias, la sincronización y las herramientas utilizadas.
4. **¿Cómo se evaluó?** Se deben presentar métricas, configuración experimental, comparación con una línea base y resultados reproducibles.
5. **¿Qué se concluye?** La conclusión debe estar conectada con evidencia, no solo con expectativas.

3. Sugerencias para elegir el tema

El tema debe ser suficientemente acotado para poder implementarse durante el semestre, pero suficientemente desafiante para requerir decisiones de computación paralela o distribuida. Se recomienda elegir un problema donde el grupo pueda obtener datos, construir una línea base secuencial y medir una mejora o un cambio de comportamiento.

Temas sugeridos:

- Procesamiento paralelo de archivos masivos: logs, transacciones, sensores, trazas, datos tabulares o texto.
- Construcción de pipelines concurrentes para lectura, limpieza, transformación y agregación de datos.
- Cálculo paralelo de estadísticas, normalización, detección de atípicos o transformaciones por bloques.
- Búsqueda de vecinos, similitud entre embeddings, recomendación o comparación masiva de vectores.
- Procesamiento de imágenes o señales mediante particionamiento espacial o por lotes.
- Simulaciones Monte Carlo, evaluación masiva de modelos o experimentos repetitivos.
- Algoritmos sobre grafos: centralidad, recorrido, componentes, PageRank simplificado o propagación.
- Comparación entre Python, OpenMP, MPI, Dask, `multiprocessing`, `concurrent.futures` u otra alternativa pertinente.
- Arquitectura híbrida que combine memoria compartida, procesos distribuidos o librerías optimizadas.

Temas no recomendados sin ajuste:

- Proyectos que solo ejecutan una librería sin explicar qué paraleliza internamente.
- Comparaciones sin línea base secuencial o sin métricas reproducibles.
- Problemas demasiado pequeños donde el overhead domina y no existe razón para paralelizar.
- Trabajos centrados solo en predicción o visualización, sin componente claro de computación paralela o distribuida.
- Propuestas que dependen de infraestructura no disponible para el grupo.

4. Desarrollo sugerido del proyecto

Etapa 1: formulación del problema

Definir el problema, el objetivo general, los objetivos específicos, el tipo de datos y la motivación técnica. En esta etapa se debe justificar por qué el problema pertenece al dominio de ciencia de datos y por qué requiere analizar rendimiento, escalabilidad, concurrencia o distribución.

Etapa 2: diagnóstico y línea base

Construir una solución inicial secuencial o una versión de referencia. Esta línea base debe permitir medir tiempo de ejecución, memoria, throughput, uso de CPU u otra métrica pertinente. Sin línea base, no es posible defender una mejora.

Etapa 3: diseño paralelo o distribuido

Seleccionar una estrategia y justificarla. El grupo debe identificar unidades de trabajo, particionamiento, granularidad, dependencias, sincronización, comunicación, reducciones, balanceo de carga y posibles cuellos de botella.

Etapa 4: implementación

Implementar un prototipo funcional. No es obligatorio construir un sistema de producción, pero el código debe ser ejecutable, comprensible y coherente con el diseño. Si se usan datos sintéticos, estos deben generarse con reglas claras y reproducibles.

Etapa 5: evaluación experimental

Medir la línea base y la solución propuesta bajo condiciones comparables. Se recomienda variar cantidad de datos, número de hilos, procesos, workers o nodos simulados, cuando corresponda.

Etapa 6: análisis crítico

Interpretar los resultados. Una mejora menor, una ausencia de mejora o incluso un empeoramiento pueden ser resultados válidos si se explican con rigor: overhead, serialización, desbalance, memoria, contención, comunicación, GIL, costo de sincronización o uso de librerías ya optimizadas.

5. Herramientas y enfoques posibles

La elección de herramientas debe estar subordinada al problema. No existe una tecnología universalmente correcta.

Enfoque	Uso recomendado	Riesgos a discutir
Python con <code>threading</code> o <code>asyncio</code>	Tareas I/O-bound, pipelines con espera, lectura concurrente o coordinación de solicitudes	GIL, bajo impacto en CPU-bound, complejidad de coordinación
<code>multiprocessing</code> o <code>ProcessPoolExecutor</code>	Tareas CPU-bound en Python, trabajo por chunks, transformaciones independientes	Serialización, copias de datos, memoria duplicada, overhead de procesos
OpenMP en C/C++	Bucles intensivos, memoria compartida, reducciones, arreglos y matrices	Condiciones de carrera, falsa compartición, balanceo, localidad de memoria

MPI o mpi4py	Distribución por procesos, mensajes, scatter/gather, problemas particionables	Costo de comunicación, sincronización, distribución de datos, depuración
Dask u otra librería de alto nivel	Procesamiento por particiones, datos mayores que memoria, flujos de trabajo escalables	Sobrecosto del scheduler, particiones mal elegidas, falsa sensación de escalabilidad
Arquitectura híbrida	Problemas que combinan datos particionados y cómputo intensivo local	Mayor complejidad, más fuentes de overhead, dificultad para atribuir mejoras

Cuadro 1: Opciones tecnológicas y criterios de uso

6. Estructura obligatoria del informe final

El informe debe ser un documento técnico, claro y reproducible. Se sugiere una extensión de 8 a 12 páginas sin contar anexos, salvo indicación distinta del docente. La extensión puede variar según la complejidad del proyecto, pero no debe reemplazar precisión por volumen.

- 1. Portada:** título del proyecto, integrantes, curso, sección, docente y fecha.
- 2. Resumen ejecutivo:** problema, estrategia, datos, principal resultado y conclusión en media página.
- 3. Introducción y motivación:** contexto del problema y relación con ciencia de datos.
- 4. Objetivo general y objetivos específicos:** metas verificables y acotadas.
- 5. Datos utilizados:** origen, tamaño, estructura, variables, formato, limpieza y restricciones. Si son sintéticos, explicar generación.
- 6. Línea base:** versión secuencial o referencia de comparación, con supuestos y métricas.
- 7. Diseño de la solución:** arquitectura, particionamiento, dependencias, sincronización, comunicación, reducciones y herramientas.
- 8. Implementación:** decisiones relevantes, comandos de ejecución, estructura del repositorio y fragmentos de código clave.
- 9. Diseño experimental:** hardware, software, variables controladas, repeticiones, tamaños de entrada y métricas.
- 10. Resultados:** tablas y gráficos con tiempos, speedup, eficiencia, memoria, throughput u otras medidas.
- 11. Análisis y discusión:** interpretación de resultados, cuellos de botella, límites, errores y comparación con expectativas.
- 12. Conclusiones:** respuesta a los objetivos, aprendizajes y condiciones bajo las cuales la estrategia cambiaría.
- 13. Referencias:** fuentes bibliográficas, documentación, datasets, repositorios o artículos utilizados.
- 14. Anexos:** instrucciones completas de ejecución, tablas adicionales, configuraciones, capturas o salidas relevantes.

Estructura mínima sugerida de entrega:

```
proyecto_semestral_inf8090/  
  informe_final.pdf  
  presentacion_final.pdf
```

```
README.md
codigo/
  src/
  scripts/
  notebooks/
datos/
  README_datos.md
resultados/
  tablas/
  graficos/
anexos/
```

7. Requisitos de reproducibilidad

El proyecto debe permitir que el docente o un par académico entienda cómo se obtuvieron los resultados. No es necesario incluir datasets enormes si existen restricciones de tamaño, pero sí se debe explicar cómo obtenerlos o generar una muestra equivalente.

- Incluir un archivo README.md con instrucciones de instalación, compilación y ejecución.
- Declarar sistema operativo, CPU, número de núcleos físicos e hilos lógicos, memoria RAM y entorno de ejecución.
- Registrar versiones de Python, compilador C/C++, OpenMP, MPI, librerías o herramientas relevantes.
- Indicar comandos usados para ejecutar la línea base y la solución paralela o distribuida.
- Usar semillas aleatorias cuando se generen datos sintéticos o experimentos aleatorios.
- Separar datos de entrada, código fuente, resultados y gráficos.
- Evitar modificar manualmente resultados experimentales. Toda tabla o gráfico debe provenir de mediciones trazables.
- Declarar cualquier uso de IA generativa, código externo, tutoriales, repositorios o documentación.

8. Métricas recomendadas

Las métricas deben seleccionarse según el problema. No todos los proyectos deben reportar todas las medidas, pero cada grupo debe justificar por qué elige las suyas.

- Tiempo total de ejecución.
- Tiempo por etapa del pipeline.
- Speedup:

$$S_p = \frac{T_1}{T_p}$$

- Eficiencia:

$$E_p = \frac{S_p}{p}$$

- Throughput: registros por segundo, archivos por minuto, comparaciones por segundo u otra unidad pertinente.
- Uso de memoria máximo o promedio.
- Escalabilidad al variar tamaño de datos o número de workers.
- Balanceo de carga entre hilos, procesos o nodos.
- Calidad de resultado si se usan aproximaciones: error, recall, diferencia porcentual, exactitud o métrica propia del dominio.

Advertencia metodológica: no se debe reportar speedup si la versión secuencial y la paralela no resuelven el mismo problema, no usan datos equivalentes o no producen resultados comparables.

9. Presentación final de máximo 15 minutos

La presentación debe comunicar el proyecto como una defensa técnica breve. El grupo debe priorizar claridad, evidencia y conclusiones. No se espera una lectura del informe ni una exposición extensa de teoría general.

Distribución sugerida del tiempo:

Bloque	Contenido	Tiempo sugerido
Problema y motivación	Qué problema se aborda, por qué importa y por qué requiere computación paralela o distribuida	2 min
Datos y línea base	Datos usados, tamaño, estructura, versión secuencial o referencia	2 min
Diseño de la solución	Arquitectura, particionamiento, dependencias, sincronización, herramientas y justificación	4 min
Resultados	Métricas, tablas, gráficos, speedup, eficiencia, memoria, throughput o calidad	4 min
Discusión y cierre	Cuellos de botella, limitaciones, conclusiones y trabajo futuro realista	2 min
Margen	Transición o preguntas breves del docente	1 min

Cuadro 2: Distribución sugerida para una presentación de 15 minutos

Estructura sugerida de diapositivas:

1. Título, integrantes y frase breve del problema.
2. Motivación y pregunta técnica principal.
3. Datos y restricciones relevantes.
4. Línea base y diagnóstico inicial.
5. Diseño paralelo o distribuido.
6. Implementación y decisiones clave.
7. Experimentos realizados.
8. Resultados principales.
9. Análisis de cuellos de botella y limitaciones.
10. Conclusiones y cierre.

Recomendaciones para la exposición:

- Presentar una historia técnica: problema, diseño, evidencia y conclusión.
- Usar gráficos simples y legibles. Un buen gráfico con interpretación vale más que muchas tablas sin lectura.
- Evitar explicar conceptos generales del curso salvo que sean necesarios para defender una decisión.
- Mostrar solo fragmentos de código que expliquen una decisión importante: particionamiento, reducción, sincronización o comunicación.
- Ensayar el tiempo. Una exposición que excede los 15 minutos puede perder claridad y evaluación.
- Distribuir roles entre integrantes, pero mantener continuidad. No debe parecer una suma de partes inconexas.

10. Criterios de evaluación

Criterio	Porcentaje	Evidencia esperada
Delimitación del problema y pertinencia para ciencia de datos	10 %	Problema claro, objetivo acotado, datos definidos y motivación real
Justificación de paralelismo, concurrencia o distribución	15 %	Diagnóstico de costo, volumen, latencia, memoria, throughput o escalabilidad
Diseño técnico de la solución	20 %	Particionamiento, dependencias, sincronización, comunicación, granularidad y herramientas
Implementación y reproducibilidad	15 %	Código ejecutable, instrucciones claras, entorno declarado y estructura ordenada
Evaluación experimental	20 %	Línea base, métricas, repeticiones, comparaciones y resultados trazables
Análisis crítico de resultados	10 %	Interpretación, limitaciones, cuellos de botella y conclusiones coherentes
Informe y presentación final	10 %	Documento claro, diapositivas precisas, defensa oral dentro de 15 minutos
Total	100 %	

Cuadro 3: Criterios sugeridos para evaluar el proyecto semestral

11. Lista de verificación antes de entregar

- El problema está claramente definido y no es solo una descripción general.
- Existe una línea base secuencial o una referencia comparable.
- La estrategia paralela o distribuida está justificada con criterios técnicos.

- Se explican particionamiento, dependencias, sincronización, comunicación o reducción, según corresponda.
- Los resultados incluyen métricas y no solo opiniones.
- Las tablas y gráficos tienen unidades, títulos y explicación.
- El informe declara entorno de ejecución, versiones y comandos.
- El código puede ejecutarse siguiendo el `README.md`.
- Las fuentes externas están citadas.
- El uso de IA generativa, si existió, está declarado.
- La presentación final dura 15 minutos o menos.

12. Integridad académica y uso de apoyo externo

El uso de documentación, ejemplos, repositorios, foros, librerías e inteligencia artificial generativa puede ser parte del proceso de aprendizaje, siempre que sea declarado y utilizado de forma crítica. El grupo es responsable de comprender, adaptar, validar y defender todo lo que entrega.

No se aceptarán proyectos que presenten código, resultados o explicaciones sin comprensión verificable. Tampoco se aceptarán entregas esencialmente idénticas entre grupos, aunque existan cambios superficiales de nombres de variables, comentarios, estructura de archivos o formato del informe.

Referencias Bibliográficas

1. Grama, A., Gupta, A., Karypis, G., & Kumar, V. (2003). *Introduction to Parallel Computing* (2nd ed.). Addison-Wesley.
2. Pacheco, P. (2011). *An Introduction to Parallel Programming*. Morgan Kaufmann.
3. Hager, G., & Wellein, G. (2010). *Introduction to High Performance Computing for Scientists and Engineers*. CRC Press.
4. Tanenbaum, A. S., & Van Steen, M. (2017). *Distributed Systems* (3rd ed.). CreateSpace.
5. OpenMP Architecture Review Board. *OpenMP API Specification*. Recuperado de <https://www.openmp.org/specifications/>
6. MPI Forum. *MPI: A Message-Passing Interface Standard*. Recuperado de <https://www.mpi-forum.org/docs/>
7. Python Software Foundation. *Python documentation*. Recuperado de <https://docs.python.org/3/>
8. Dask. *Dask documentation*. Recuperado de <https://docs.dask.org>
9. Fuentes oficiales adicionales, datasets, notas de clase y documentación técnica citadas explícitamente por cada grupo.